

LAPORAN PRAKTIKUM PERTEMUAN 3
PEMROGRAMAN MOBILE



Dosen Pengampu:
Jefry Sunupurwa Asri, S.Kom., M.Kom.

Disusun Oleh:
Ramsay Abelson 20240801042

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
TANGERANG
2026

Tujuan

- Mengambil input pengguna dengan `TextField` dan `TextEditingController`
- Membuat form dengan validasi memakai `Form`, `TextFormField`, dropdown, dan checkbox
- Menjelaskan keterbatasan `setState` ketika data dipakai banyak halaman
- Menerapkan state management dasar dengan `ChangeNotifier` dan paket `Provider`
- Membedakan penggunaan `context.watch` dan `context.read`

Bagian A: Input Dasar dengan `TextField`

Kode (`lib/main.dart`):

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 3',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const InputPage(),
    );
  }
}

class InputPage extends StatefulWidget {
  const InputPage({super.key});

  @override
  State<InputPage> createState() => _InputPageState();
}

class _InputPageState extends State<InputPage> {
  final _controller = TextEditingController();
  String _salam = '';

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

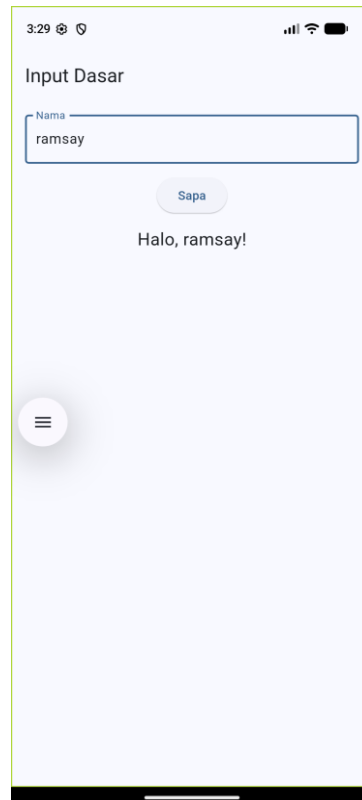
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Input Dasar')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
```

```

        onPressed: () {
          setState(() => _salam = 'Halo, ${_controller.text}!');
        },
        child: const Text('Sapa'),
      ),
      const SizedBox(height: 12),
      Text(_salam, style: const TextStyle(fontSize: 20)),
    ],
  ),
),
);
}
}

```

Hasil:



Gambar 1. Hasil Bagian A

Penjelasan:

- `TextField` menerima input teks, nilainya dibaca lewat `TextEditingController`
- Saat tombol Sapa ditekan, `setState()` mengisi `_salam` dengan teks dari `_controller.text`
- `dispose()` melepas controller saat halaman ditutup agar tidak terjadi kebocoran memori

Bagian B: Form dengan Validasi

Kode:

```

class FormPage extends StatefulWidget {
  const FormPage({super.key});

  @override
  State<FormPage> createState() => _FormPageState();
}

class _FormPageState extends State<FormPage> {

```

```

final _formKey = GlobalKey<FormState>();
final _nama = TextEditingController();
final _email = TextEditingController();
String? _jurusan;
bool _setuju = false;

@override
void dispose() {
  _nama.dispose();
  _email.dispose();
  super.dispose();
}

void _ kirim() {
  if (_formKey.currentState!.validate()) {
    final jurusan = _jurusan ?? '-';
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Terdaftar: ${_nama.text} ($jurusan)'),
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Form Pendaftaran')),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _nama,
            decoration: const InputDecoration(
              labelText: 'Nama lengkap',
              border: OutlineInputBorder(),
            ),
            validator: (v) =>
              (v == null || v.trim().isEmpty) ? 'Nama wajib diisi' : null,
          ),
          const SizedBox(height: 12),
          TextFormField(
            controller: _email,
            keyboardType: TextInputType.emailAddress,
            decoration: const InputDecoration(
              labelText: 'Email',
              border: OutlineInputBorder(),
            ),
            validator: (v) {
              if (v == null || !v.contains('@')) return 'Email tidak valid';
              return null;
            },
          ),
          const SizedBox(height: 12),
          DropdownButtonFormField<String>(
            decoration: const InputDecoration(
              labelText: 'Jurusan',
              border: OutlineInputBorder(),
            ),
            items: const [
              DropdownMenuItem(value: 'TI', child: Text('Teknik Informatika')),
              DropdownMenuItem(value: 'SI', child: Text('Sistem Informasi')),
              DropdownMenuItem(value: 'TE', child: Text('Teknik Elektro')),
            ],
            onChanged: (v) => setState(() => _jurusan = v),
            validator: (v) => v == null ? 'Pilih jurusan' : null,
          ),
          CheckboxListTile(
            title: const Text('Saya menyetujui ketentuan'),
            value: _setuju,
            controlAffinity: ListTileControlAffinity.leading,
            onChanged: (v) => setState(() => setuju = v ?? false),
          ),
          ElevatedButton(
            onPressed: _setuju ? _ kirim : null,

```

```

        child: const Text('Daftar'),
      ),
    ],
  ),
),
);
}
}

```

Lalu home : di MyApp diubah menjadi `const FormPage()`.

Hasil:

Gambar 2. Form terisi dengan benar

Gambar 3. Pesan error email tidak valid

Penjelasan:

- Form dengan `GlobalKey<FormState>` memungkinkan semua isian divalidasi sekaligus lewat `validate()`
- validator pada `TextFormField` mengembalikan pesan error, atau null jika valid
- Pada Gambar 3, email tanpa tanda @ memunculkan pesan merah "Email tidak valid"
- `DropdownButtonFormField` dipakai untuk memilih jurusan, `CheckboxListTile` untuk persetujuan
- Tombol Daftar hanya aktif jika checkbox dicentang (`onPressed: _setuju ? _kirim : null`)
- Jika semua valid, `SnackBar` menampilkan pesan terdaftar

Bagian C: Mengapa Butuh State Management?

- Pada Bagian A dan B, state hidup di satu halaman dan cukup memakai `setState`
- Pada aplikasi daftar tugas, halaman daftar dan halaman tambah harus berbagi data yang sama

- Mengoper data lewat constructor dan callback antar halaman cepat menjadi rumit
- Solusinya: pindahkan data ke satu objek bersama yang diakses kedua halaman, yaitu dengan Provider

Bagian D: Aplikasi Daftar Tugas dengan Provider

Kode (setelah `flutter pub add provider`):

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;
  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai => _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Tugas',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const TugasPage(),
    );
  }
}

class TugasPage extends StatelessWidget {
  const TugasPage({super.key});

  @override
  Widget build(BuildContext context) {
```

```

final model = context.watch<TugasModel>();

return Scaffold(
  appBar: AppBar(
    title: Text('Tugas (${model.jumlahSelesai}/${model.items.length}')),
  ),
  body: ListView.builder(
    itemCount: model.items.length,
    itemBuilder: (context, i) {
      final t = model.items[i];
      return ListTile(
        leading: Checkbox(
          value: t.selesai,
          onChanged: (_) => context.read<TugasModel>().toggle(i),
        ),
        title: Text(
          t.judul,
          style: TextStyle(
            decoration: t.selesai ? TextDecoration.lineThrough : null,
          ),
        ),
        trailing: IconButton(
          icon: const Icon(Icons.delete),
          onPressed: () => context.read<TugasModel>().hapus(i),
        ),
      );
    },
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () {
      Navigator.push(
        context,
        MaterialPageRoute(builder: (_) => const TambahPage()),
      );
    },
    child: const Icon(Icons.add),
  ),
);
}
}

class TambahPage extends StatefulWidget {
  const TambahPage({super.key});

  @override
  State<TambahPage> createState() => _TambahPageState();
}

class _TambahPageState extends State<TambahPage> {
  final _controller = TextEditingController();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _simpan() {
    final judul = _controller.text.trim();
    if (judul.isEmpty) return;
    context.read<TugasModel>().tambah(judul);
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tambah Tugas')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(

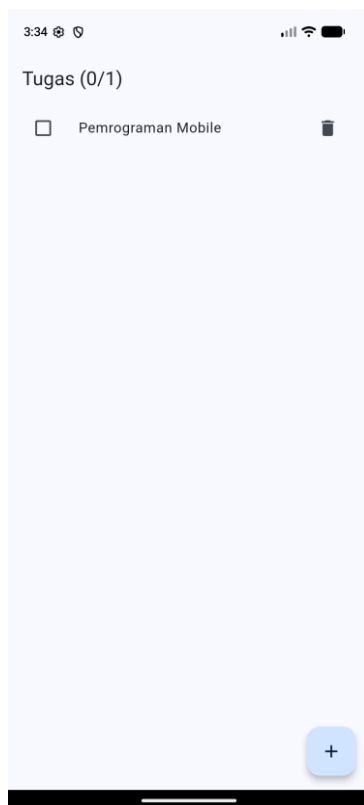
```

```

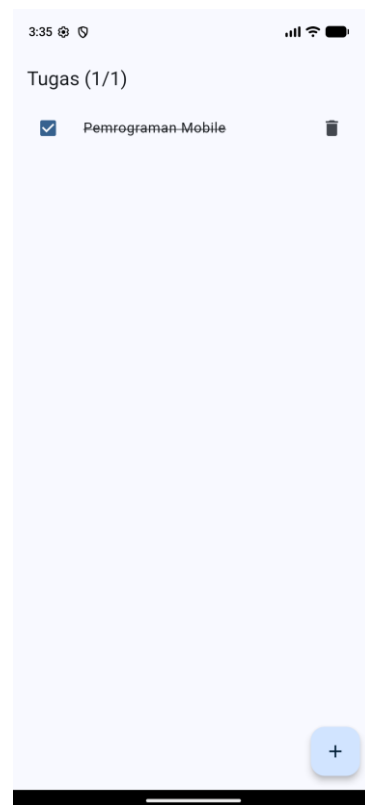
        controller: _controller,
        autofocus: true,
        decoration: const InputDecoration(
          labelText: 'Judul tugas',
          border: OutlineInputBorder(),
        ),
        onSubmitted: (_) => _simpan(),
      ),
      const SizedBox(height: 12),
      ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
    ],
  ),
),
);
}
}

```

Hasil:



Gambar 4. Tugas ditambahkan (0/1)



Gambar 5. Tugas dicentang (1/1) dan dicoret

Penjelasan:

- TugasModel adalah turunan ChangeNotifier yang menyimpan daftar tugas
- notifyListeners() memberi tahu widget agar rebuild setiap data berubah
- ChangeNotifierProvider ditaruh di atas MyApp agar model bisa diakses semua halaman
- context.watch dipakai di build() supaya tampilan ikut berubah
- context.read dipakai di callback seperti onChanged dan onPressed karena hanya perlu memanggil method
- Halaman Tambah menyimpan tugas ke model yang sama, lalu langsung muncul di halaman daftar
- Mencentang tugas mengubah judul AppBar menjadi (1/1) dan mencoret teks

Latihan Mandiri

Kode:

```
// ===== Latihan 2: method hapusSelesai() pada TugasModel =====
void hapusSelesai() {
  _items.removeWhere((t) => t.selesai);
  notifyListeners();
}

// ===== Latihan 2 & 4: perubahan pada TugasPage =====
AppBar: AppBar(
  title: Text('Tugas (${model.jumlahSelesai}/${model.items.length}')),
  actions: [
    IconButton(
      icon: const Icon(Icons.delete_sweep),
      tooltip: 'Hapus yang selesai',
      onPressed: () => context.read<TugasModel>().hapusSelesai(),
    ),
  ],
),
// Latihan 4: teks saat daftar kosong
body: model.items.isEmpty
  ? const Center(child: Text('Belum ada tugas'))
  : ListView.builder(
    // ... sama seperti sebelumnya
  ),

// ===== Latihan 1 & 3: TambahPage memakai Form + validasi + Snackbar =====
class _TambahPageState extends State<TambahPage> {
  final _formKey = GlobalKey<FormState>();
  final _controller = TextEditingController();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _simpan() {
    // Latihan 1: validasi minimal 3 karakter
    if (!_formKey.currentState!.validate()) return;

    final messenger = ScaffoldMessenger.of(context);
    context.read<TugasModel>().tambah( controller.text.trim());
    Navigator.pop(context);

    // Latihan 3: Snackbar setelah disimpan
    messenger.showSnackBar(
      const SnackBar(content: Text('Tugas ditambahkan')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tambah Tugas')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: _controller,
                autofocus: true,
                decoration: const InputDecoration(
                  labelText: 'Judul tugas',
                  border: OutlineInputBorder(),
                ),
                validator: (v) {
                  if (v == null || v.trim().length < 3) {

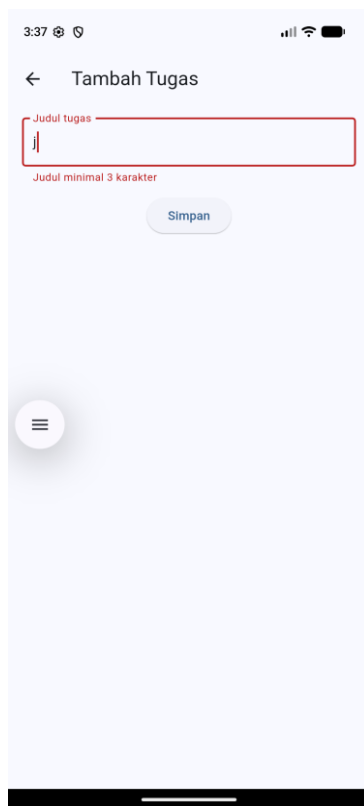
```

```

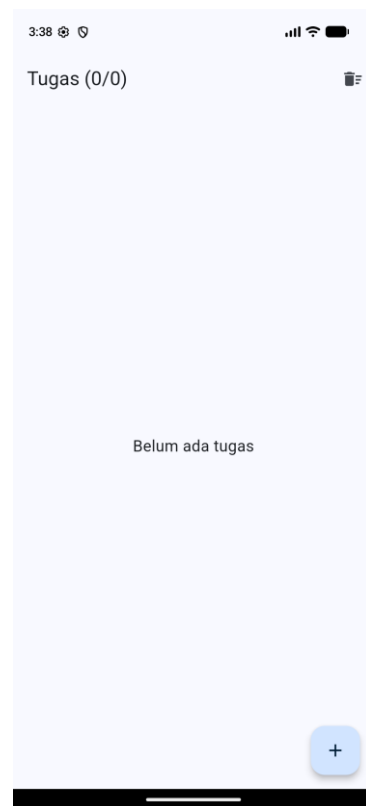
        return 'Judul minimal 3 karakter';
      }
      return null;
    },
    onFieldSubmitted: (_) => _simpan(),
  ),
  const SizedBox(height: 12),
  ElevatedButton(
    onPressed: _simpan,
    child: const Text('Simpan'),
  ),
),
),
),
),
),
);
}
}

```

Hasil:



Gambar 6. Validasi judul minimal 3 karakter



Gambar 7. Tampilan saat daftar kosong

Penjelasan:

1. **Validasi judul:** TambahPage diubah memakai Form dan TextFormField, judul kurang dari 3 karakter menampilkan error (Gambar 6)
2. **hapusSelesai():** method baru di TugasModel memakai removeWhere, dipanggil dari tombol Icons.delete_sweep di AppBar (terlihat di Gambar 7)
3. **SnackBar:** setelah tugas disimpan, muncul pesan "Tugas ditambahkan". messenger diambil sebelum Navigator.pop agar tetap valid
4. **Daftar kosong:** jika model.items.isEmpty, tampil teks "Belum ada tugas" di tengah layar (Gambar 7)

Tugas: Daftar Belanja

Kode:

```
// Jalankan dulu: flutter pub add provider
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

// ===== Model Data =====
class Barang {
  String nama;
  int jumlah;
  String kategori;
  bool dibeli;

  Barang(this.nama, this.jumlah, this.kategori, {this.dibeli = false});
}

// ===== State (ChangeNotifier) =====
class BelanjaModel extends ChangeNotifier {
  final List<Barang> _items = [];

  List<Barang> get items => List.unmodifiable(_items);

  int get belumDibeli => _items.where((b) => !b.dibeli).length;

  void tambah(String nama, int jumlah, String kategori) {
    _items.add(Barang(nama, jumlah, kategori));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].dibeli = !_items[index].dibeli;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}

const daftarKategori = ['Makanan', 'Minuman', 'Kebersihan', 'Lainnya'];

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => BelanjaModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Belanja',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.green, useMaterial3: true),
      home: const BelanjaPage(),
    );
  }
}

// ===== Halaman Daftar =====
class BelanjaPage extends StatelessWidget {
```

```

const BelanjaPage({super.key});

@override
Widget build(BuildContext context) {
  final model = context.watch<BelanjaModel>();

  return Scaffold(
    appBar: AppBar(
      title: Text('Belanja (${model.belumDibeli} belum dibeli)'),
    ),
    body: model.items.isEmpty
      ? const Center(child: Text('Belum ada barang'))
      : ListView.builder(
          itemCount: model.items.length,
          itemBuilder: (context, i) {
            final b = model.items[i];
            return ListTile(
              leading: Checkbox(
                value: b.dibeli,
                onChanged: (_) => context.read<BelanjaModel>().toggle(i),
              ),
              title: Text(
                b.nama,
                style: TextStyle(
                  decoration: b.dibeli ? TextDecoration.lineThrough : null,
                ),
              ),
              subtitle: Text('${b.jumlah} • ${b.kategori}'),
              trailing: IconButton(
                icon: const Icon(Icons.delete),
                onPressed: () => context.read<BelanjaModel>().hapus(i),
              ),
            );
          },
        ),
    floatingActionButton: FloatingActionButton(
      onPressed: () {
        Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => const TambahBarangPage()),
        );
      },
      child: const Icon(Icons.add),
    ),
  );
}

// ===== Halaman Form Tambah =====
class TambahBarangPage extends StatefulWidget {
  const TambahBarangPage({super.key});

  @override
  State<TambahBarangPage> createState() => _TambahBarangPageState();
}

class _TambahBarangPageState extends State<TambahBarangPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _jumlah = TextEditingController();
  String? _kategori;

  @override
  void dispose() {
    _nama.dispose();
    _jumlah.dispose();
    super.dispose();
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
  }
}

```

```

        context.read<BelanjaModel>().tambah(
            _nama.text.trim(),
            int.parse(_jumlah.text.trim()),
            _kategori!,
        );
        Navigator.pop(context);
    }

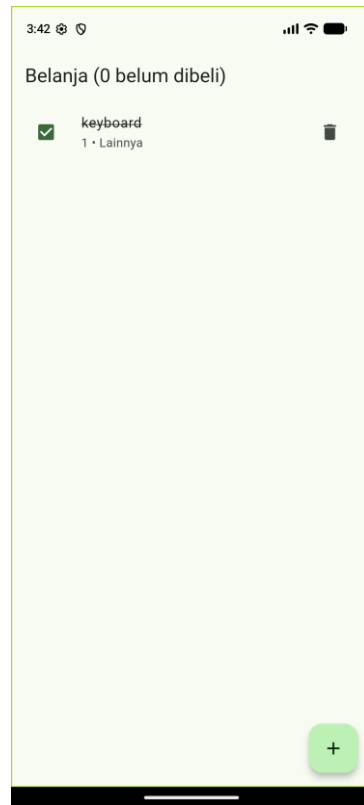
    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(title: const Text('Tambah Barang')),
            body: Form(
                key: _formKey,
                child: ListView(
                    padding: const EdgeInsets.all(16),
                    children: [
                        TextFormField(
                            controller: _nama,
                            decoration: const InputDecoration(
                                labelText: 'Nama barang',
                                border: OutlineInputBorder(),
                            ),
                        ),
                        validator: (v) =>
                            (v == null || v.trim().isEmpty) ? 'Nama barang wajib diisi' : null,
                    ),
                    const SizedBox(height: 12),
                    TextFormField(
                        controller: _jumlah,
                        keyboardType: TextInputType.number,
                        decoration: const InputDecoration(
                            labelText: 'Jumlah',
                            border: OutlineInputBorder(),
                        ),
                    ),
                    validator: (v) {
                        if (v == null || v.trim().isEmpty) {
                            return 'Jumlah wajib diisi';
                        }
                        final angka = int.tryParse(v.trim());
                        if (angka == null || angka <= 0) {
                            return 'Jumlah harus lebih dari 0';
                        }
                        return null;
                    },
                ),
                    const SizedBox(height: 12),
                    DropdownButtonFormField<String>(
                        decoration: const InputDecoration(
                            labelText: 'Kategori',
                            border: OutlineInputBorder(),
                        ),
                    ),
                    items: daftarKategori
                        .map((k) => DropdownMenuItem(value: k, child: Text(k)))
                        .toList(),
                    onChanged: (v) => setState(() => _kategori = v),
                    validator: (v) => v == null ? 'Pilih kategori' : null,
                ),
                    const SizedBox(height: 16),
                    ElevatedButton(
                        onPressed: _simpan,
                        child: const Text('Simpan'),
                    ),
                ],
            ),
        );
    }
}

```

Hasil:



Gambar 8. Form tambah barang dengan pesan error validasi



Gambar 9. Halaman daftar belanja

Penjelasan:

- BelanjaModel (ChangeNotifier) menyimpan daftar Barang dan dibagikan lewat ChangeNotifierProvider
- belumDibeli menghitung barang yang belum dicentang, ditampilkan di AppBar
- Form Tambah Barang memvalidasi nama (wajib), jumlah (wajib, angka lebih dari 0), dan kategori (dropdown)
- Pada Gambar 8, jumlah 0 memunculkan pesan "Jumlah harus lebih dari 0"
- Halaman daftar menampilkan nama, jumlah, dan kategori. Barang bisa dicentang (dicoret) dan dihapus
- Pada Gambar 9, barang keyboard sudah dicentang sehingga AppBar menampilkan "0 belum dibeli"

Jawaban Pertanyaan Refleksi

1. **Mengapa TextEditingController harus di-dispose?** Agar memori yang dipakai controller dilepas saat halaman ditutup dan tidak terjadi kebocoran memori.
2. **Kapan cukup memakai setState, dan kapan beralih ke Provider?** setState cukup untuk state yang hanya dipakai satu widget atau satu halaman. Beralih ke Provider jika data dipakai banyak halaman.
3. **Apa yang terjadi bila notifyListeners() lupa dipanggil?** Data di model berubah, tetapi tampilan tidak ikut diperbarui karena widget yang berlangganan tidak diberi tahu untuk rebuild.

4. **Mengapa di dalam onPressed memakai context.read, bukan context.watch?** Di dalam callback kita hanya perlu mengakses model sekali untuk memanggil method, tanpa perlu rebuild. `context.watch` hanya untuk di dalam `build()`.

Kesimpulan

- `TextField` dan `TextEditingController` dipakai untuk mengambil input, `Form` dan `validator` untuk validasi
- `setState` cocok untuk state lokal, sedangkan data lintas halaman lebih rapi dengan `Provider`
- `notifyListeners()` wajib dipanggil agar tampilan ikut berubah
- `context.watch` untuk `build()`, `context.read` untuk callback